

Bitvector Rank, Select, Access

Foundations of succinct data structures

Bitvectors

Does this bitvector have a "meaning?"

What if its name was `is_prime`? 🤔

How might we query it?

E.g. next-highest-prime

E.g. designing a 2-universal hash,
we want smallest prime (leftmost 1)
greater than some number

0
1
1
0
1
0
1
0
0
0
1
0
1
⋮

Bitvectors

What if the bitvector represented membership in some set?

Why might want to "navigate" it?

Say we are counting 1s to estimate cardinality

Might want to "jump" between 1s,
ask how spaced out they are
(k^{th} minimum value)

$$\vdots$$

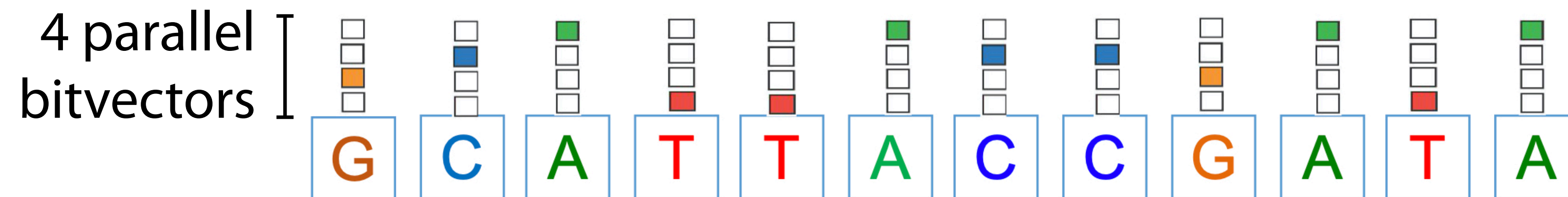
Bitvectors

Might represent words in a document

0	abinoam
1	abiogenesis
0	abiological
0	abiosis
0	abiotic
0	abiotically
0	abiotrophy
0	abirritate
0	abishag
0	abit
0	abitibi
0	abiu
1	abject
⋮	

Bitvectors

Could be a "one-hot" encoding of string



Navigating bitvectors = navigating the occurrences of characters in the string

Bitvectors

How do we navigate / query bitvectors?

Proposal: "RSA" (Rank, Select, Access)

0
1
1
0
1
0
1
0
0
0
1
0
1

Bitvectors

$$B . \text{access}(i) = B[i]$$

Conceptually trivial, but
harder if we compress B

(more later)

Indexing starts at 0

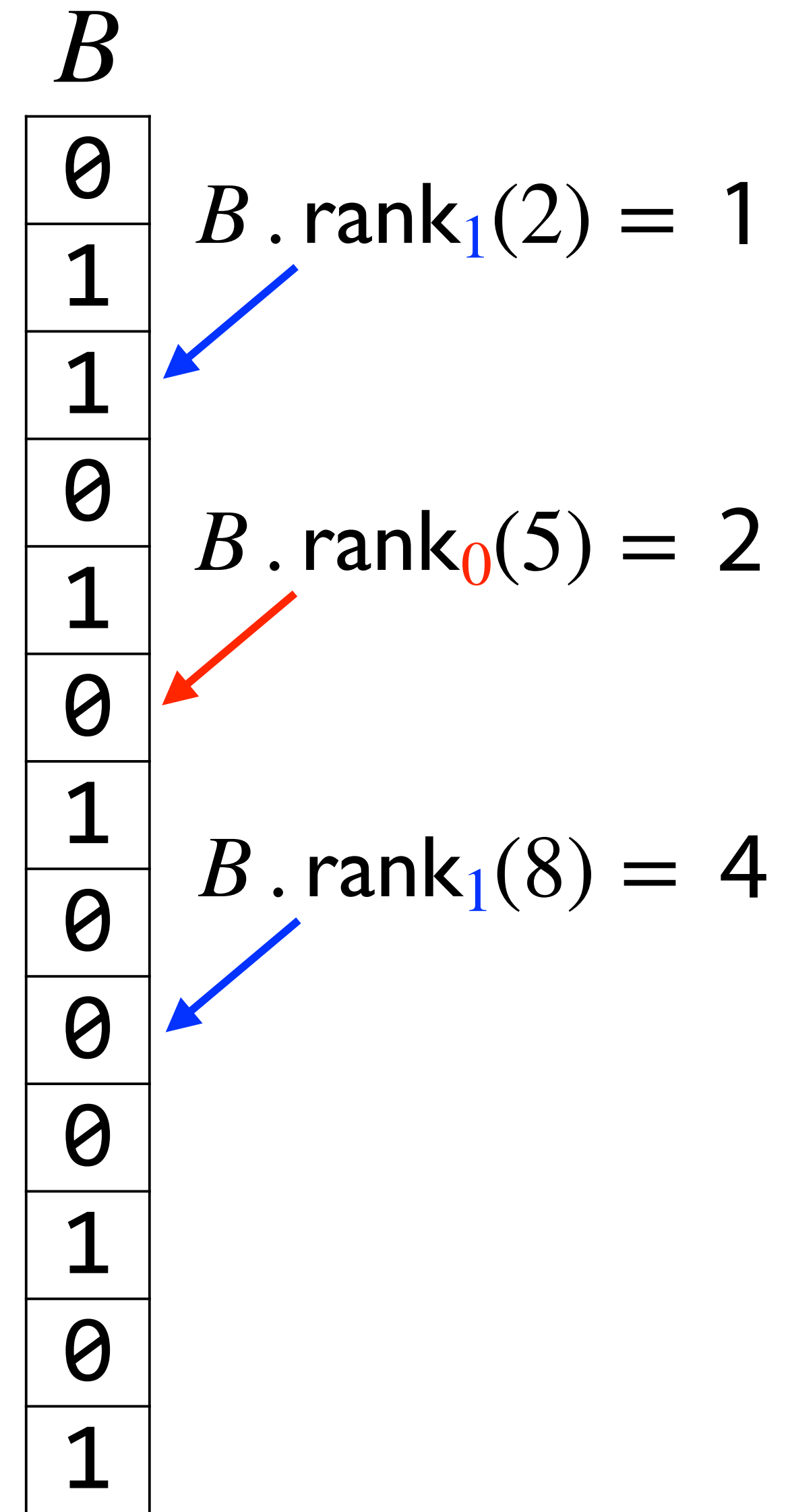
B	
0	
1	
1	$\leftarrow B . \text{access}(2) = 1$
0	
1	
0	
1	
0	$\leftarrow B . \text{access}(7) = 0$
0	
0	
1	
0	
1	

Bitvectors

$$B . \text{rank}_1(i) = \sum_{j=0}^{i-1} B[j]$$

$$B . \text{rank}_0(i) = i - B . \text{rank}_1(i)$$

Rank counts up to
but not including offset i



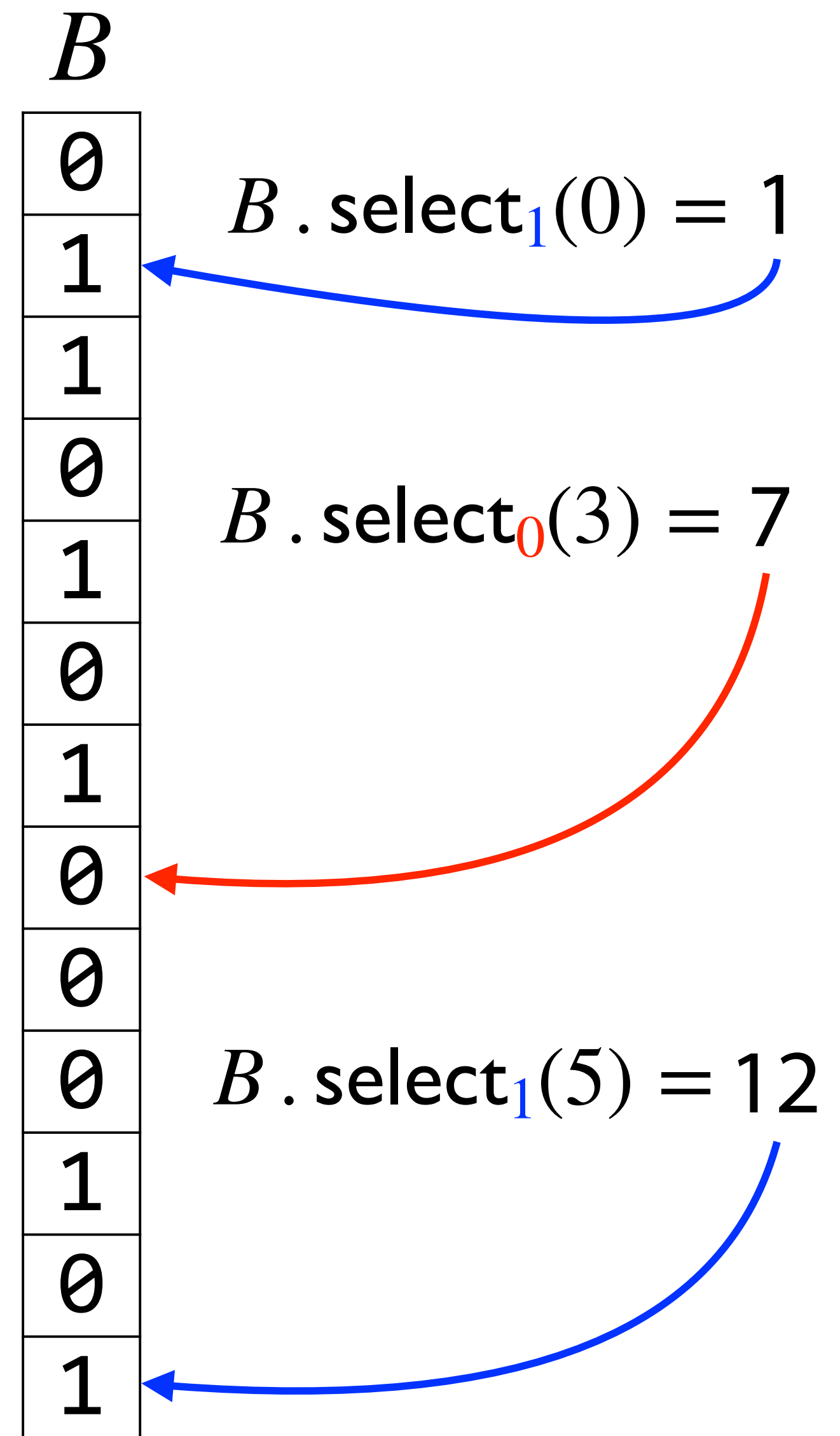
Bitvectors

$$B . \text{select}_1(i) =$$

$$\max \{ j \mid B . \text{rank}_1(j) = i \}$$

$$B . \text{select}_0(i) =$$

$$\max \{ j \mid B . \text{rank}_0(j) = i \}$$



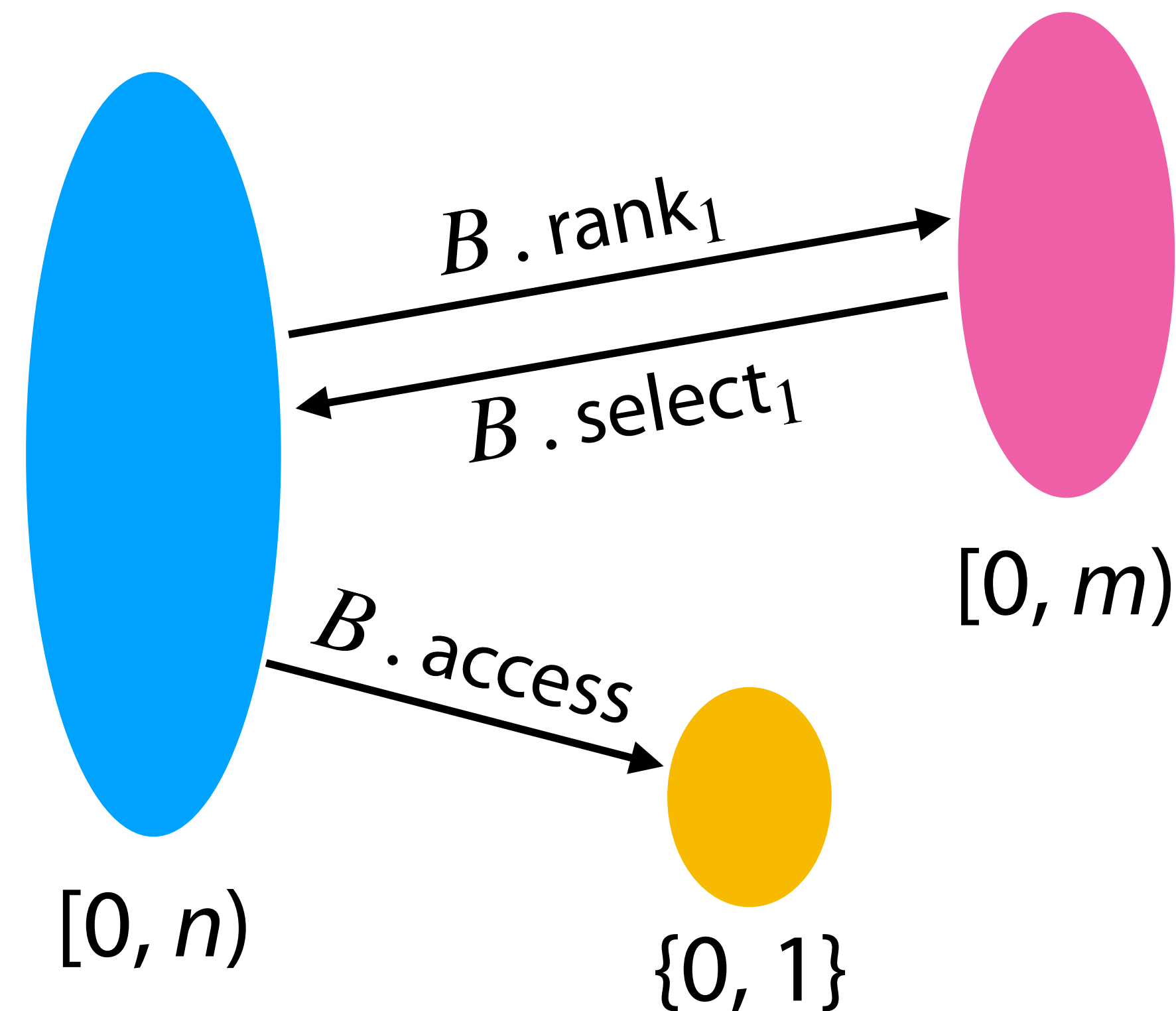
Bitvectors

$B . \text{access}(\dots)$

$B . \text{rank}(\dots)$

$B . \text{select}(\dots)$

Let $|B| = n$ and let m equal
the number of set bits



Bitvectors

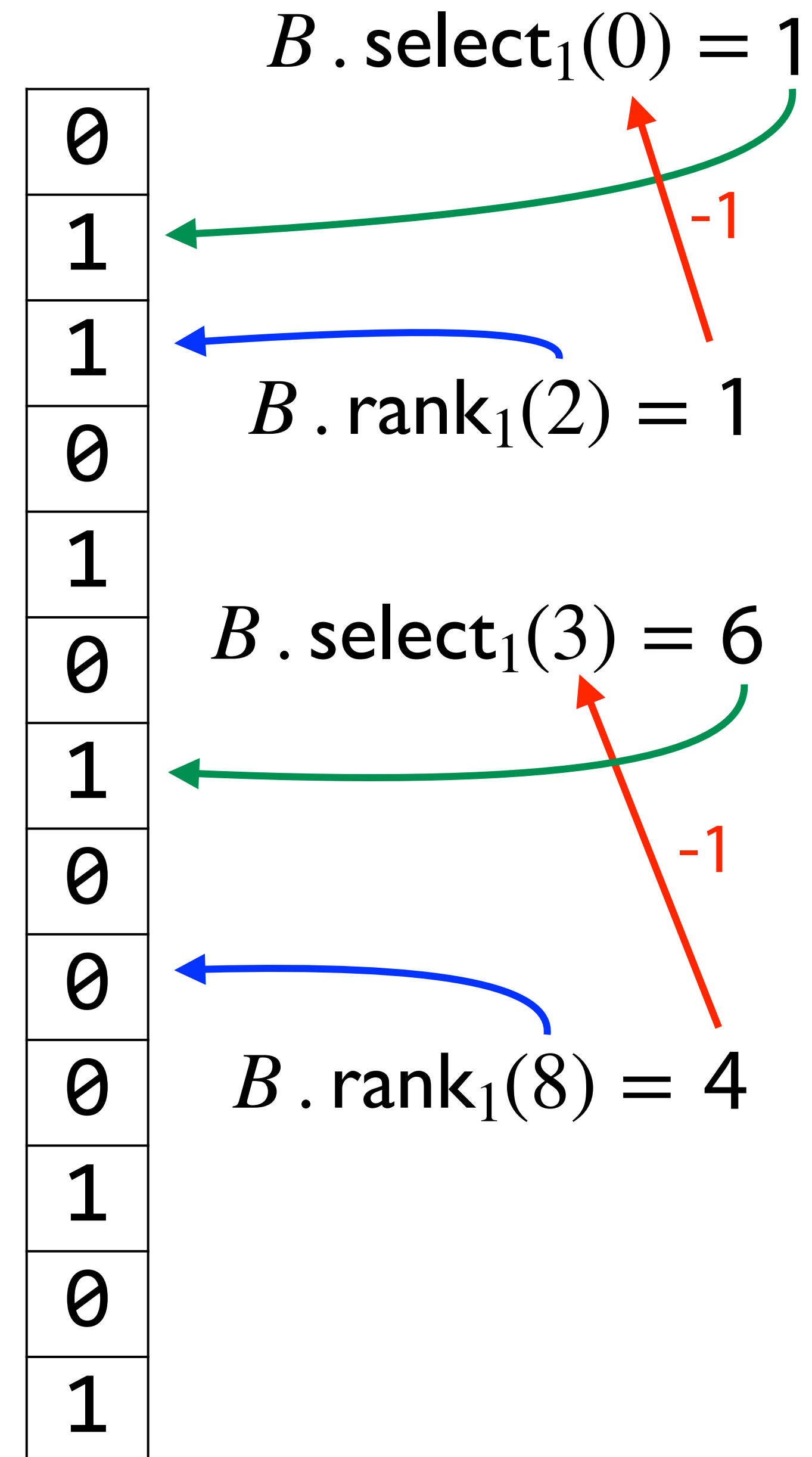
What does this do?

$$B . \text{select}_1(B . \text{rank}_1(i) - 1)$$

$$B . \text{rank}_1(i) = \sum_{j=0}^{i-1} B[j]$$

$$B . \text{select}_1(i) = \max \{ j \mid B . \text{rank}_1(j) = i \}$$

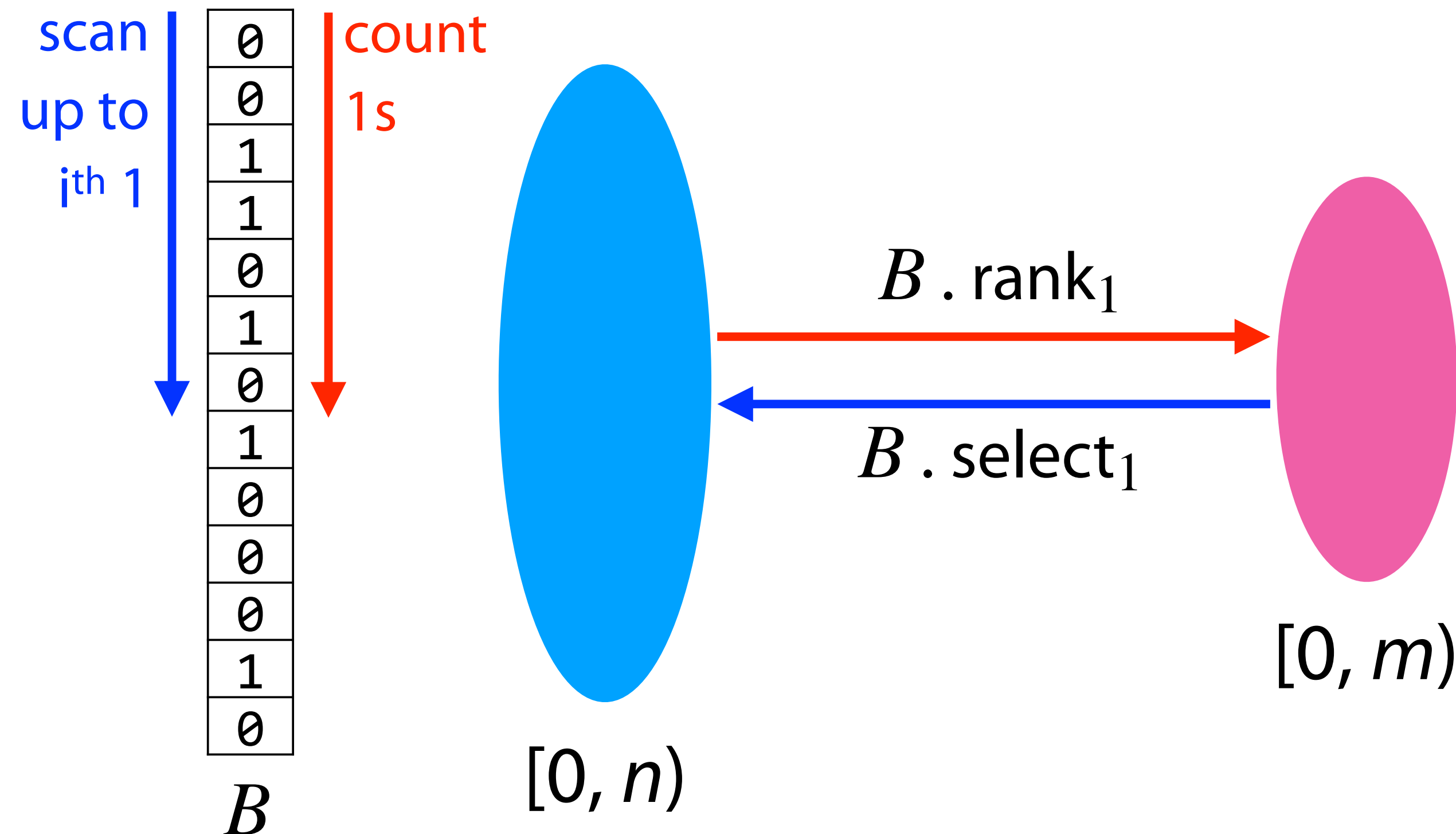
Gives offset of next-earliest set bit -- *predecessor*



Bitvectors

How to implement $B.\text{rank}_1$ & $B.\text{select}_1$?

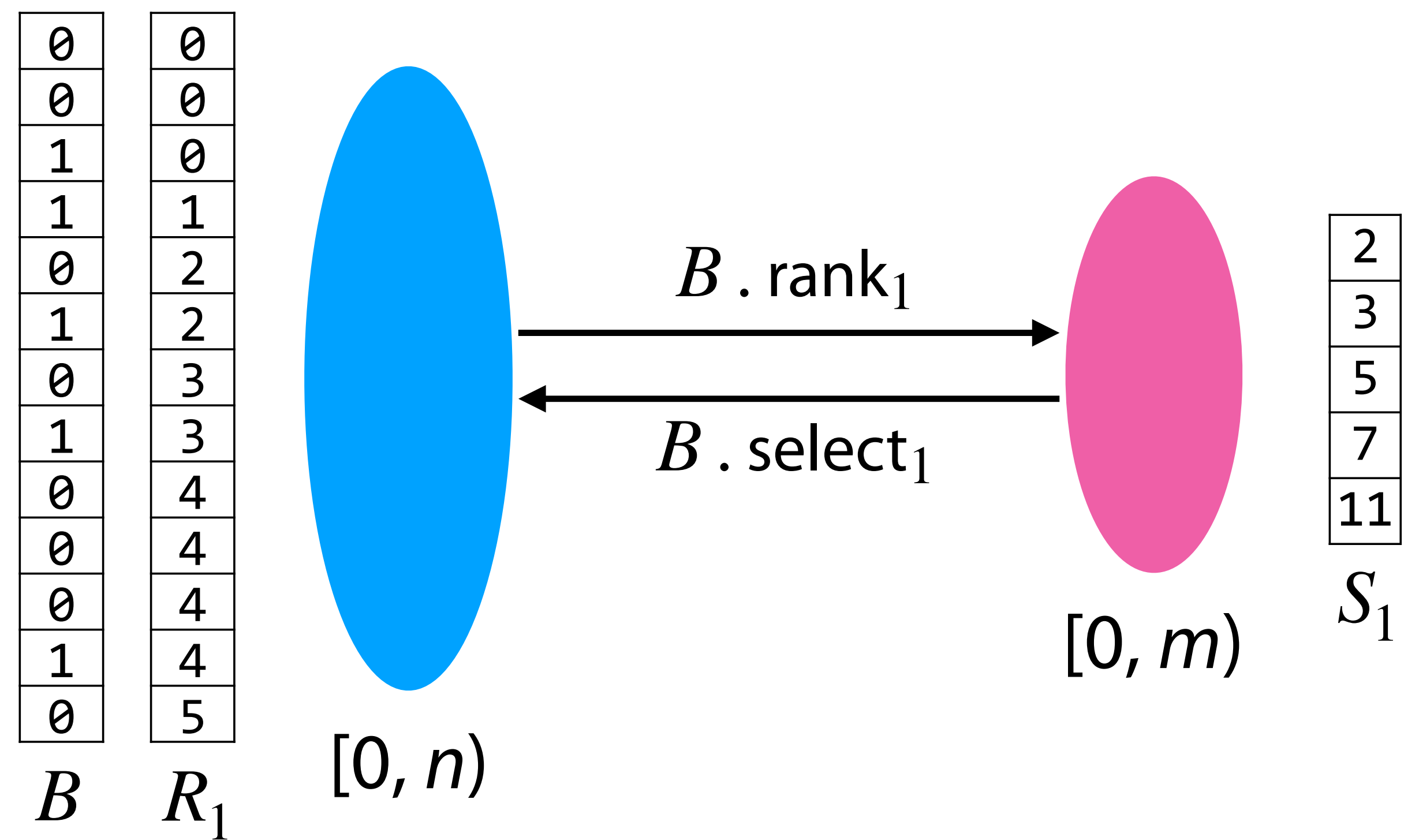
Idea 0: linear scans over B



Can we be more efficient?

Bitvectors

Idea 1: Pre-calculate all answers



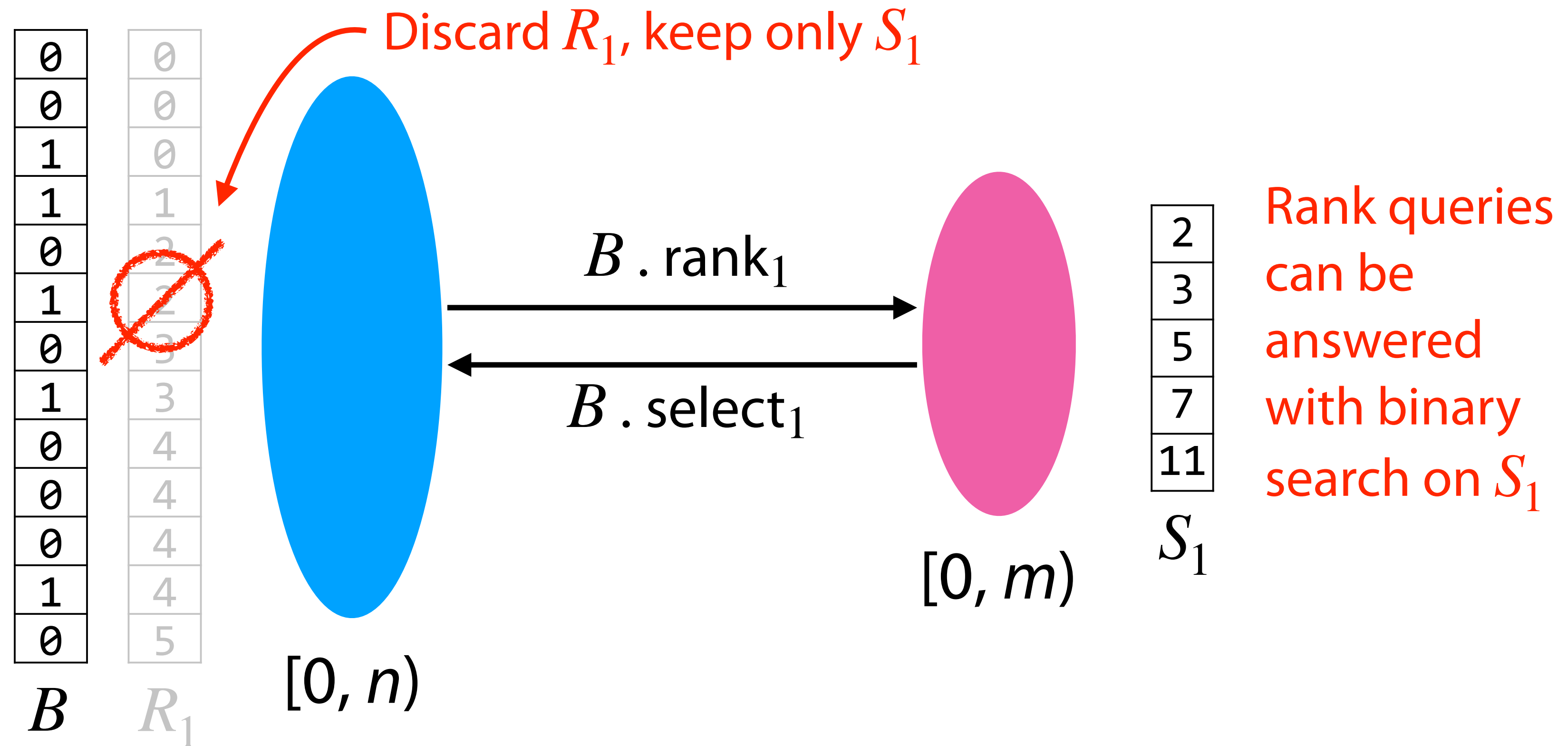
Bitvectors

Idea 1: Pre-calculate all answers

	Time	Space (bits)	Note
$B . \text{access}$	$O(1)$	n	Lookup
$B . \text{select}_1$	$O(1)$	$O(m \log n)$	Pre-calculate S_1
$B . \text{rank}_1$	$O(1)$	$O(n \log m)$	Pre-calculate R_1

Bitvectors

Idea 2: Pre-calculate all answers for $B \cdot \text{select}_1$



$O(m \log n)$ bits. $B \cdot \text{rank}_1$ is $O(\log m)$ time.

Bitvectors

Idea 2: Pre-calculate all answers for B . select_1

	Time	Space (bits)	Note
B . access	$O(1)$	n	Lookup
B . select_1	$O(1)$	$O(m \log n)$	Pre-calculate S_1
B . rank_1	$O(\log m)$	$O(m \log n)$	Binary search on S_1

Bitvectors

Coming soon:

	Time	Space (bits)	Note
$B . \text{access}$	$O(1)$	n	Lookup
$B . \text{select}_1$	$O(1)$	$\tilde{O}(n)$	?    ?
$B . \text{rank}_1$	$O(1)$	$\tilde{O}(n)$	?    ?